



Université Évre Paris-Saclay

Département IBGBI

Une intelligence artificielle imbattable au jeu de dames

Monte Carlo Tree Search amélioré

Travail d'Étude et de Recherche

Encadrant : Blaise Hanczar

Étudiants : Hajar El Barhdadi 20252434@etud.univ-evry.fr
Ferial Beldjilali 20254774@etud.univ-evry.fr
Wissam Sadok 20254797@etud.univ-evry.fr

Année universitaire 2025 – 2026

Table des matières

1	Introduction	3
1.1	État de l’art	3
1.2	Notre approche	4
1.3	Organisation du rapport	4
2	Le jeu de dames comme problème d’IA	4
2.1	Règles essentielles	4
2.2	Complexité et enjeux algorithmiques	5
2.3	Représentation interne du plateau	5
3	Monte Carlo Tree Search	5
3.1	Idée fondamentale	5
3.2	Les quatre phases de l’algorithme	6
3.2.1	Sélection	6
3.2.2	Expansion	6
3.2.3	Simulation	6
3.2.4	Rétropropagation	7
3.3	Structure de l’arbre et choix final	7
4	Nos contributions : heuristiques et évaluation	7
4.1	Rollouts guidés	8
4.2	Évaluation positionnelle à score continu	9
4.2.1	Score matériel et positionnel	9
4.2.2	Transformation sigmoïde	9
5	Implémentation	10
5.1	Architecture du projet	10
5.2	Interface graphique	10
5.3	Moteur de règles	11
5.4	Configuration	11
6	Démonstration interactive	11
7	Expériences et résultats	12
7.1	Protocole	12
7.2	Évaluation 1 : l’effet du nombre d’itérations	12
7.3	Évaluation 2 : MCTS amélioré * contre MCTS classique *	13
8	Limites et perspectives	15
8.1	Ce que notre approche ne résout pas encore	15

8.2 Pistes d'amélioration	15
9 Conclusion	15
A Constantes globales — <code>utilities.py</code>	17
B Point d'entrée — <code>main.py</code>	17
C Structure d'un nœud MCTS — classe <code>MCTSNode</code>	17
D Rollouts guidés — méthode <code>_smart_random_move</code>	18
E Évaluation positionnelle — méthode <code>evaluate_board</code>	19
F Boucle de recherche — méthode <code>mcts_search</code>	19

1 Introduction

Le jeu de dames * est l'un de ces jeux dont les règles paraissent simples, mais dont la profondeur stratégique demande des années de pratique pour être réellement maîtrisée. À première vue, cela ne semble pas représenter un défi majeur pour une machine : après tout, un ordinateur peut explorer des millions de positions par seconde. En réalité, l'espace des états est tellement vaste — de l'ordre de 5×10^{20} positions légales — qu'une exploration exhaustive devient rapidement impossible.

Ce TER est né d'une question simple : peut-on construire, à partir d'un algorithme général de recherche arborescente *, une intelligence artificielle * capable de jouer efficacement au jeu de dames *, au point de rivaliser avec un bon joueur humain ? Notre réponse, que ce rapport cherche à démontrer, est oui — à condition d'enrichir cet algorithme avec des heuristiques * capables de guider intelligemment les simulations * sans pour autant complexifier excessivement le système d'évaluation *.

1.1 État de l'art

Les premières approches en intelligence artificielle * appliquées aux jeux combinatoires reposent principalement sur l'algorithme minimax *, souvent amélioré par l'élagage alpha-bêta *. Cette méthode permet d'explorer efficacement l'arbre de jeu en supprimant les branches inutiles, mais reste fortement dépendante d'une fonction d'évaluation heuristique * de qualité, dont la conception devient complexe pour des jeux à forte profondeur stratégique comme le jeu de dames *.

Pour répondre à cette limitation, des méthodes basées sur des simulations * ont été introduites, notamment le Monte Carlo Tree Search * (MCTS). Cet algorithme estime la valeur d'une position en effectuant des simulations aléatoires * à partir de celle-ci. Introduit par Coulom [2] et formalisé par Browne et al. [1], MCTS * a permis des avancées importantes dans de nombreux jeux complexes grâce à sa capacité à équilibrer exploration * et exploitation * sans dépendre fortement d'une heuristique explicite *.

Cependant, les simulations purement aléatoires * peuvent introduire du bruit et produire des estimations instables lorsque le budget de calcul est limité. Pour y remédier, plusieurs travaux ont proposé d'orienter les simulations * par des connaissances a priori. Baier et Winands [3] montrent ainsi qu'une politique de simulation biaisée par des heuristiques simples * améliore nettement la convergence de MCTS. Par ailleurs, Chaslot *et al.* [4] ont introduit des stratégies progressives pour équilibrer exploration * et exploitation*, en ajustant dynamiquement le poids de l'exploration *.

Plus récemment, des approches hybrides ont été proposées, combinant MCTS * avec des méthodes d'apprentissage automatique. Des systèmes tels que AlphaGo [6]

utilisent des réseaux de neurones pour guider la recherche et remplacer partiellement les simulations aléatoires * par des politiques apprises. Ces travaux montrent l'intérêt d'intégrer des connaissances a priori pour améliorer la qualité des décisions.

Dans cette continuité, notre travail propose une version améliorée de MCTS * intégrant des heuristiques * simples afin de guider les simulations * et d'affiner l'évaluation des positions.

1.2 Notre approche

Nous avons choisi de construire notre solution à partir de cette base en y intégrant deux améliorations principales : des *rollouts guidés*, qui orientent les simulations * vers des coups stratégiquement plus cohérents, et une *évaluation positionnelle* à score continu, qui permet d'obtenir une estimation plus fine de la qualité d'une position qu'un simple résultat victoire/défaite.

Ces contributions, associées à un moteur de jeu complet avec interface graphique interactive, constituent le cœur de notre travail.

1.3 Organisation du rapport

Nous présentons d'abord les règles du jeu et la structure générale du problème (section 2), puis l'algorithme MCTS * dans sa version classique (section 3). La section 4 détaille les heuristiques * et améliorations proposées. La section 5 décrit l'implémentation réalisée. Nous présentons ensuite une démonstration interactive de l'algorithme (section 6). Enfin, la section 7 présente les résultats expérimentaux, suivis d'une discussion sur les limites et perspectives (section 8) et d'une conclusion (section 9).

2 Le jeu de dames comme problème d'IA

2.1 Règles essentielles

Le jeu de dames que nous avons implémenté correspond à la variante anglaise (*English draughts* ou *American checkers*). Il se joue sur les 32 cases foncées d'un damier 8×8 . Chaque joueur dispose de 12 pions au départ, placés sur les trois premières rangées de son côté. Un pion se déplace en diagonale vers l'avant, d'une case à la fois. La règle la plus contraignante — et stratégiquement la plus riche — est la **capture obligatoire** : si un pion adverse peut être sauté, le joueur doit le faire, et doit enchaîner si d'autres captures sont possibles depuis la nouvelle position.

Un pion qui atteint la dernière rangée adverse devient **roi**. Dans les dames anglaises *, un roi * se déplace d'une seule case en diagonale, mais dans les quatre directions

(avant et arrière), ce qui augmente sa mobilité sans atteindre la puissance des « rois volants » d'autres variantes. La partie s'arrête lorsqu'un joueur n'a plus de pièces ou plus aucun coup légal disponible.

2.2 Complexité et enjeux algorithmiques

Ce qui rend le jeu de dames * intéressant d'un point de vue algorithmique, c'est la tension entre sa simplicité apparente et la profondeur de son espace de recherche. Schaeffer *et al.* ont prouvé en 2007 que le jeu est résolu [5], mais leur programme Chinook * a nécessité des décennies de calcul distribué. Dans notre contexte — un jeu en temps réel, sur une machine personnelle — l'enjeu n'est pas d'être optimal au sens absolu, mais d'être *suffisamment bon* dans un temps de calcul raisonnable. C'est exactement le compromis que MCTS * est conçu pour gérer.

2.3 Représentation interne du plateau

Dans notre implémentation, le plateau est une matrice 8×8 d'entiers. Chaque case contient l'une des cinq valeurs définies dans `utilities.py` (voir annexe A) :

Valeur	Signification	Constante
0	Case vide	—
1	Pion noir	BLACK_PIECE
2	Pion rouge	RED_PIECE
3	Roi noir	BLACK_KING
4	Roi rouge	RED_KING

Ce choix est délibérément minimaliste : copier l'état du plateau pour une simulation * revient à copier un tableau d'entiers, ce qui est extrêmement rapide en Python * — et cela devient crucial lorsqu'on enchaîne des milliers de simulations * par coup.

3 Monte Carlo Tree Search

3.1 Idée fondamentale

MCTS * est un algorithme de prise de décision qui apprend à jouer en simulant des parties. Plutôt que d'évaluer chaque position avec une fonction d'heuristique * minutieusement calibrée — ce qui demande une expertise approfondie du jeu — MCTS * joue mentalement des milliers de parties au brouillon depuis la position courante et observe lesquelles se terminent par une victoire. À force de simulations *, certains coups apparaissent statistiquement meilleurs que d'autres, et c'est celui qui se démarque le plus que l'IA * choisit.

L'élégance de cette approche tient à sa généralité : elle s'applique à n'importe quel jeu dont on peut simuler les parties, sans avoir besoin de comprendre les subtilités stratégiques — du moins en théorie. En pratique, comme nous allons le voir, orienter intelligemment ces simulations * fait toute la différence.

3.2 Les quatre phases de l'algorithme

MCTS * organise sa recherche en un cycle de quatre opérations répétées jusqu'à épuisement du budget de calcul (ici, un nombre fixe d'itérations).

3.2.1 Sélection

On part de la racine de l'arbre — l'état courant du jeu — et on descend vers les feuilles en choisissant à chaque étape le fils le plus prometteur. Ce choix repose sur la formule UCT * (*Upper Confidence Bound for Trees*), qui équilibre deux impératifs contradictoires :

$$\text{UCT}(i) = \bar{X}_i + C \cdot \sqrt{\frac{\ln N}{n_i}} \quad (1)$$

Le premier terme, $\bar{X}_i$, représente le taux de succès observé du nœud * i : c'est l'*exploitation*, qui pousse à revisiter les coups qui ont bien marché par le passé. Le second terme est une mesure d'incertitude : plus un nœud * est peu visité (n_i petit) par rapport à son parent (N grand), plus cette valeur est élevée. C'est l'*exploration*, qui évite de négliger des coups au potentiel inexploré. La constante $C \approx 1.41$ règle l'équilibre entre les deux. Un nœud * jamais visité reçoit une valeur infinie, garantissant qu'il sera exploré au moins une fois.

3.2.2 Expansion

Lorsque la descente atteint un nœud * insuffisamment développé, on lui ajoute de nouveaux fils correspondant aux coups légaux non encore essayés. C'est ainsi que l'arbre * grandit au fil des itérations, en s'étendant prioritairement dans les directions prometteuses.

3.2.3 Simulation

Depuis le nœud nouvellement ajouté, on joue une partie complète jusqu'à un état terminal — victoire, défaite ou match nul — en choisissant les coups selon une politique de simulation. Dans notre version améliorée, cette politique n'est pas purement aléatoire mais guidée par des heuristiques * (voir section 4.1).

3.2.4 Rétropropagation

Le résultat de la simulation remonte le chemin parcouru jusqu'à la racine : chaque nœud * traversé voit son compteur de visites incrémenté et son score mis à jour. Dans notre implémentation, ce score n'est pas binaire mais continu dans $[0, 1]$, ce qui rend les estimations plus stables et plus nuancées (voir section 4.2).

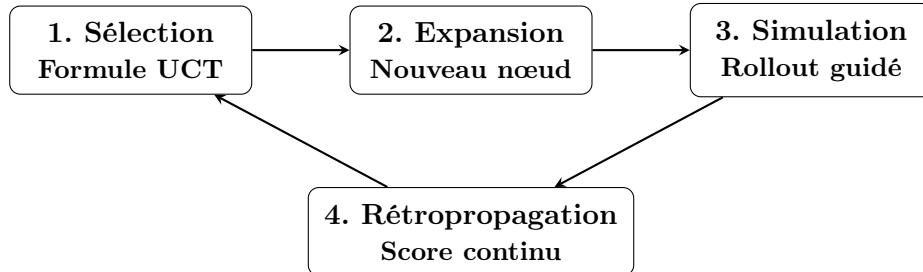


FIGURE 1 – Les quatre phases cycliques de MCTS *. À chaque itération, on descend dans l'arbre (sélection), on l'étend (expansion), on simule une partie (simulation) et on remonte le résultat (rétropropagation).

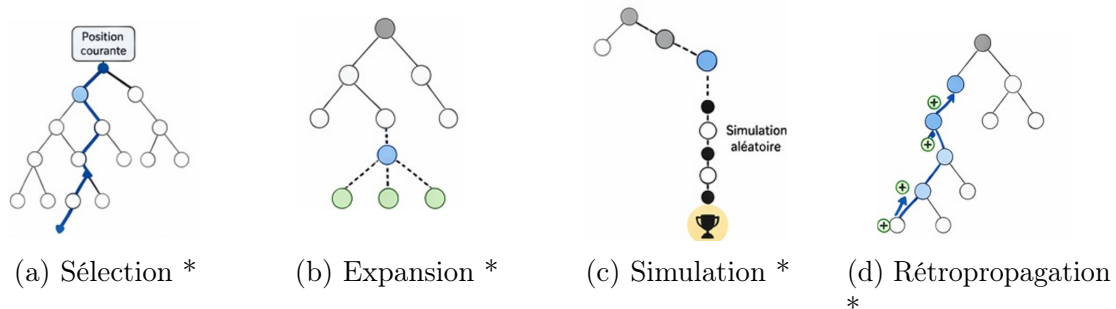


FIGURE 2 – Les quatre étapes de l'algorithme MCTS

3.3 Structure de l'arbre et choix final

Chaque nœud * mémorise l'état du plateau, le joueur dont c'est le tour, le coup qui y a mené depuis le père, et deux compteurs : le nombre de visites et le cumul des scores de simulation. La structure complète est présentée en annexe C.

Une fois le budget épuisé, l'IA choisit le fils de la racine ayant été le *plus visité* — et non celui au meilleur score. Ce choix est délibéré : un nœud * très visité est celui que l'UCT * a jugé prometteur de façon répétée, ce qui le rend plus robuste face à la variance naturelle des simulations *.

4 Nos contributions : heuristiques et évaluation

Le MCTS classique * souffre d'un problème bien documenté : ses simulations purement aléatoires * génèrent beaucoup de bruit. Une partie jouée au hasard ne ressemble

à aucune partie réelle — les pièces reculent sans raison, les captures évidentes sont ignorées, le centre du plateau est délaissé. Le résultat est une estimation de la valeur d'un coup qui peut être très éloignée de la réalité, surtout avec un budget de calcul limité.

Nos deux contributions visent à corriger ce défaut en injectant un minimum de connaissance stratégique dans l'algorithme, à deux endroits précis : lors des simulations (*rollouts guidés* *) et lors de l'évaluation du résultat (*score continu par sigmoïde* *).

4.1 Rollouts guidés

L'idée est simple : plutôt que de choisir uniformément parmi tous les coups légaux lors d'une simulation *, on attribue un score à chacun d'eux et on tire aléatoirement parmi le meilleur 50 % des candidats. On conserve ainsi une part de stochasticité — indispensable pour que l'arbre * explore des lignes variées — tout en éliminant les coups les plus clairement sous-optimaux.

Le score d'un coup est calculé à partir de quatre critères cumulatifs. Premièrement, les captures sont traitées comme une priorité absolue : si une prise est possible, seuls les coups de capture entrent en compétition, conformément aux règles du jeu. Deuxièmement, l'**avancement** récompense les pièces qui progressent vers la promotion : un pion qui avance gagne des points de score, ce qui pousse naturellement la simulation * à aller de l'avant. Troisièmement, le **contrôle du centre** est valorisé via un bonus inversement proportionnel à la distance des colonnes centrales (3 et 4), qui offrent plus de flexibilité tactique que les bords. Enfin, les **rois** * reçoivent un bonus supplémentaire reflétant leur mobilité accrue.

Score heuristique des coups non capturants :

- avancement vers la promotion : $+r$ pour les noirs (plus r est grand, plus le pion avance), $+(7 - r)$ pour les rouges ;
- contrôle du centre : $+(2 - |c - 3,5| \times 0,5)$;
- bonus roi : $+1$ si la pièce déplacée est un roi *.

Pourquoi ça marche. En éliminant les coups manifestement sous-optimaux, les rollouts guidés * produisent des simulations qui ressemblent davantage à de vraies parties. L'arbre MCTS * reçoit un signal plus fiable, converge plus vite vers les bonnes lignes de jeu, et prend de meilleures décisions pour un même nombre d'itérations. C'est l'équivalent de donner à l'algorithme un instinct de base du jeu.

L'implémentation complète est disponible en annexe D.

4.2 Évaluation positionnelle à score continu

La seconde contribution concerne la façon dont le résultat d'une simulation * est mesuré et propagé. Dans un MCTS classique *, chaque simulation * se conclut par une victoire (valeur 1) ou une défaite (valeur 0). Cette binarité est brutale : elle ne distingue pas entre une position perdue après avoir longtemps dominé et une position où on n'avait aucune chance.

Notre approche remplace cette valeur binaire par un **score continu dans** $[0, 1]$, calculé à partir d'une évaluation multi-critères de l'état du plateau en fin de simulation *. Cela fournit une information plus fine sur la qualité des positions et réduit la variance

4.2.1 Score matériel et positionnel

Pour chaque pièce présente, on calcule une contribution selon sa nature et sa position. Un pion vaut 10 points de base, auxquels s'ajoutent jusqu'à 3 points d'avancement (normalisé sur 7 rangées) et un bonus de centrage entre 0 et 1. Un roi vaut 20 points de base plus un double bonus de centrage, reflétant sa valeur tactique supérieure. C'est la différence nette entre le score rouge et le score noir qui compte.

TABLE 1 – Critères de l'évaluation positionnelle et leur contribution.

Critère	Formule	Contribution
Valeur de base	pion = 10, roi = 20	principale
Avancement	$3 \times r/7$ (noir) ou $3 \times (7 - r)/7$ (rouge)	+3 max
Centre	$1 - c - 3,5 /7$	$\in [0, 1]$
Défense dernière ligne	+1 si pion sur sa ligne de départ	bonus
Roi au centre	+2 $\times$ centre	bonus

4.2.2 Transformation sigmoïde

La différence brute peut prendre n'importe quelle valeur sur $\mathbb{R}$. Pour la ramener dans $[0, 1]$, on applique une fonction sigmoïde * :

$$\text{résultat} = \frac{1}{1 + e^{-\Delta/15}} \quad (2)$$

où Δ est la différence de scores en faveur du joueur MCTS *. Le paramètre 15 règle la sensibilité : une légère avance donne un score proche de 0.5 ; une domination écrasante pousse le score vers 1. L'arbre apprend ainsi à distinguer les positions légèrement avantageuses des positions clairement gagnantes, et calibre ses explorations en conséquence.

Le code complet de cette fonction se trouve en annexe E.

5 Implémentation

5.1 Architecture du projet

Le projet est organisé autour de plusieurs fichiers Python *, chacun ayant un rôle précis.

- `utilities.py` regroupe les constantes partagées (annexe A).
- `game.py` est le fichier central : il contient le moteur des règles, la logique MCTS * avec nos heuristiques *, et l'interface graphique.
- `mcts_classic.py` implémente un MCTS classique * (simulations entièrement aléatoires). Cette version sert de référence pour comparer les performances de notre approche améliorée.
- `evaluation.py` automatise les expériences et les tournois entre les différentes IA *. Il génère les statistiques et graphiques utilisés dans l'analyse des résultats.
- `run_one_game.py` et `main.py` (annexe B) sont des points d'entrée pour lancer une partie. Ils permettent de configurer le nombre d'itérations du MCTS * avant d'exécuter la boucle principale du jeu.

Cette architecture volontairement simple évite la dispersion entre de nombreux fichiers et facilite la compréhension du flux d'exécution — un critère important pour un projet académique destiné à être relu et évalué.

5.2 Interface graphique

L'interface est développée avec `pygame` et s'affiche dans une fenêtre redimensionnable. Le plateau occupe la partie gauche, avec une bande de coordonnées (colonnes 0–7 en bas, lignes 0–7 à gauche) qui facilite le suivi des positions. Quand le joueur humain sélectionne une pièce, les cases accessibles s'allument en vert pour les déplacements simples et en rouge pour les captures, rendant l'interface immédiatement lisible.

À droite du plateau, un panneau latéral affiche en temps réel le tour en cours, le compteur de coups, un historique des derniers déplacements, et un **slider interactif** permettant d'ajuster le nombre d'itérations MCTS * entre 100 et 4 000 sans relancer le programme. Cette fonctionnalité s'est révélée précieuse pendant les tests : on observe en direct comment la qualité du jeu de l'IA * évolue avec le budget de calcul.

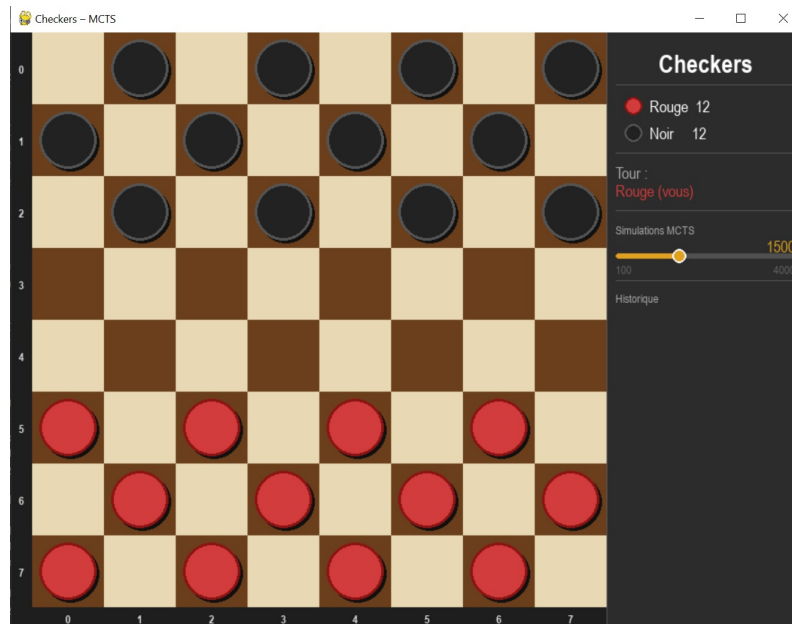


FIGURE 3 – Interface du jeu de dames.

5.3 Moteur de règles

Le moteur gère l'ensemble des règles officielles : déplacements légaux de pions et de rois, capture obligatoire, prises multiples enchaînées récursivement, promotion automatique, et détection de fin de partie. Un point technique notable concerne les prises multiples : après chaque capture, le moteur vérifie immédiatement si la pièce peut en réaliser une autre depuis sa nouvelle position, et lors des simulations MCTS *, les captures multiples sont gérées récursivement, ce qui assure le respect des règles tout en maintenant la cohérence des rollouts.

5.4 Configuration

Le seul paramètre exposé à l'utilisateur final est le nombre d'itérations, accessible via le constructeur de `Game` ou le slider de l'interface. Le fichier `main.py` documente les plages de performance typiques : entre 500 et 1 000 itérations pour un niveau débutant avec réponse quasi-instantanée, entre 1 000 et 2 000 pour un bon compromis, et au-delà de 2 000 pour un jeu avancé au prix d'un léger délai.

La boucle principale de `mcts_search *`, qui orchestre les quatre phases décrites précédemment, est disponible en annexe F.

6 Démonstration interactive

Afin de rendre le projet accessible et pédagogique, nous avons développé une démonstration interactive disponible à l'adresse :

<https://mctshjr.netlify.app/>



Scannez ce code pour accéder directement à la démonstration.

Cette démonstration visualise en temps réel le fonctionnement de l'algorithme :

- construction progressive de l'arbre de recherche * noeud par noeud* ;
- mise en évidence du chemin de sélection uct* à chaque itération ;
- affichage des rollouts* guidés et de leurs résultats ;
- visualisation de la backprop* et de la mise à jour des statistiques.

Elle permet à l'utilisateur d'observer concrètement comment l'arbre MCTS * converge vers les meilleurs coups et constitue un outil pédagogique efficace pour comprendre l'algorithme sans lire le code source.

7 Expériences et résultats

7.1 Protocole

Nous avons mené deux séries d'évaluations complémentaires. La première quantifie l'impact brut du nombre d'itérations, toutes choses égales par ailleurs. La seconde mesure le gain réel apporté par nos heuristiques face à un MCTS classique * à simulations aléatoires . Pour éliminer les biais liés au premier joueur, chaque configuration est testée en alternant quel agent joue en premier sur les parties successives.

7.2 Évaluation 1 : l'effet du nombre d'itérations

Une IA forte * à 1 200 itérations a affronté successivement des IA plus faibles * à 100, 300, 600 et 800 itérations, à raison de 20 parties par configuration (80 parties au total).

Résultat	Parties	Pourcentage
Victoires IA forte	40 / 80	50 %
Victoires IA faible	28 / 80	35 %
Matchs nuls	12 / 80	15 %

L'IA forte * domine nettement, confirmant que le nombre d'itérations reste le premier facteur de performance dans MCTS *. Cependant, le taux non négligeable de victoires des IA faibles * (35 %) illustre une propriété intéressante : même avec peu d'itérations,

une IA guidée par nos heuristiques * tire parti de la stochasticité de MCTS * pour surprendre une IA plus puissante *. Ses simulations *, même rapides, restent orientées stratégiquement — et cela compte.

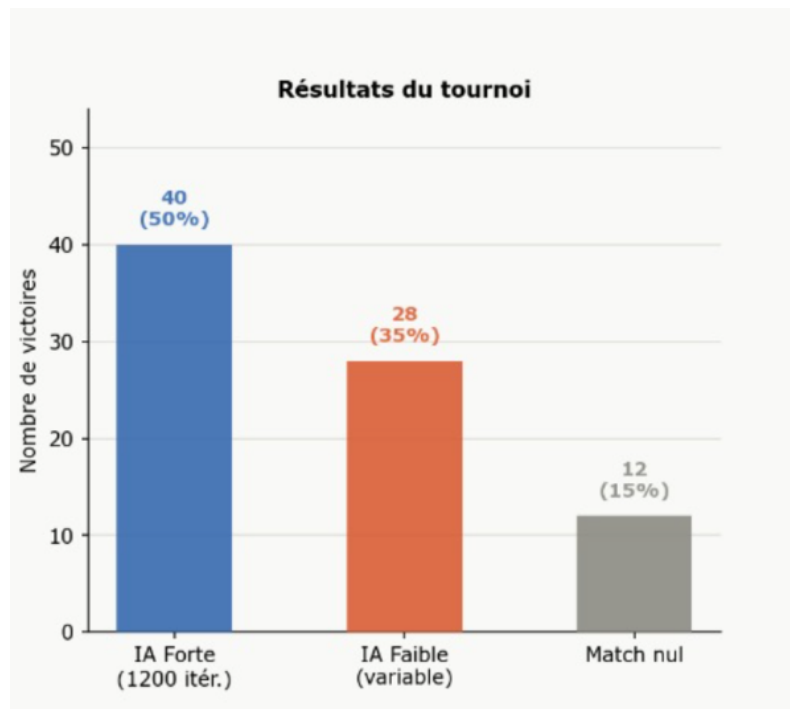


FIGURE 4 – Distribution des résultats lors des tournois IA forte * (1 200 itérations) contre IA plus faibles * (100–800 itérations), sur 80 parties au total.

7.3 Évaluation 2 : MCTS amélioré * contre MCTS classique *

La seconde évaluation compare directement notre MCTS amélioré * à un MCTS classique * à rollouts purement aléatoires *, sur 25 parties par configuration (1 000, 2 000 et 3 000 itérations).

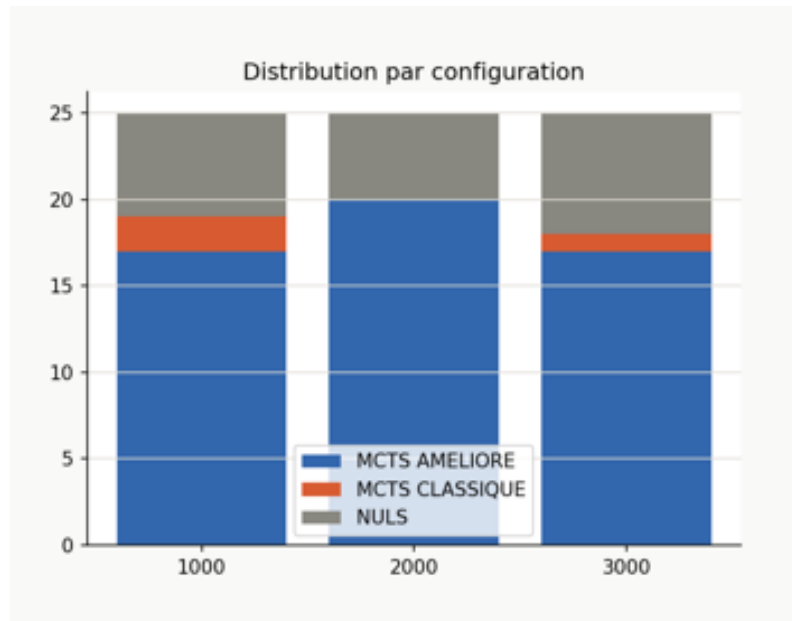


FIGURE 5 – Résultats comparatifs MCTS amélioré vs MCTS classique sur 25 parties par configuration. Les résultats montrent l'évolution des victoires à 1 000, 2 000 et 3 000 itérations.

TABLE 2 – Résultats détaillés de l'évaluation 2 (25 parties par configuration).

Itérations	MCTS Amélioré	MCTS Classique	Nuls	Winrate Amélioré
1 000	17	2	6	68,0 %
2 000	20	0	5	80,0 %
3 000	17	1	7	68,0 %
Total	54	3	18	72,0 %

Les résultats sont sans ambiguïté. Le MCTS amélioré * gagne davantage de parties sur *toutes* les configurations testées, et l'écart se creuse à mesure que le nombre d'itérations augmente : à 3 000 itérations, la supériorité de notre version est particulièrement marquée. Les matchs nuls restent stables quelle que soit la configuration, ce qui indique que nos heuristiques * se traduisent directement en victoires supplémentaires plutôt qu'en simples réductions de défaites.

Lecture des résultats. L'écart croissant avec le nombre d'itérations s'explique ainsi : avec peu d'itérations, les deux versions ont un arbre * peu développé et la chance domine. Avec plus d'itérations, l'arbre est plus profond, et la qualité des simulations commence à faire la différence — les rollouts guidés * orientent l'exploration vers les bonnes lignes, et l'évaluation sigmoïde * discrimine mieux entre positions proches. L'heuristique * a le temps de s'exprimer.

8 Limites et perspectives

8.1 Ce que notre approche ne résout pas encore

La première limite, inhérente à MCTS * lui-même, est sa **nature stochastique**. Même avec nos heuristiques *, la variance des résultats reste significative sur un nombre restreint de parties : il arrive qu’une IA plus faible * batte une IA plus forte * simplement parce que la chance a souri à ses simulations *. Des techniques comme RAVE * (*Rapid Action Value Estimation*) permettraient de réduire cette variance en partageant les statistiques entre nœuds similaires, sans coût de calcul excessif.

Nos pondérations d’évaluation constituent une autre limite. Les coefficients qui régissent la valeur d’un pion, d’un roi ou d’une case centrale ont été fixés empiriquement. Ils fonctionnent raisonnablement bien mais ne sont pas optimaux. Une approche de **TD-learning** — où ces coefficients seraient ajustés au fil des parties — pourrait significativement améliorer la précision de l’évaluation.

Enfin, notre implémentation est entièrement **séquentielle**, alors que MCTS * se parallélise naturellement : chaque simulation * est indépendante des autres. Une parallélisation multi-cœurs multiplierait le nombre effectif de simulations * dans le même temps de calcul, sans aucune modification de la logique de l’algorithme.

8.2 Pistes d’amélioration

Les perspectives les plus réalistes à court terme sont au nombre de trois. La première est l’intégration de **RAVE**, qui accélérerait la convergence de l’arbre *. La deuxième est la parallélisation par *threads* Python *, qui doublerait ou triplerait le budget de simulation * à temps de réponse égal. La troisième, plus ambitieuse, consisterait à remplacer notre fonction d’évaluation heuristique * par un **réseau de neurones léger** entraîné sur des parties auto-jouées, à la façon d’AlphaZero * — à une échelle bien plus modeste, mais en conservant l’esprit de l’approche.

9 Conclusion

Ce projet nous a confrontés à un problème classique en intelligence artificielle * : comment prendre de bonnes décisions dans un espace de possibilités trop vaste pour être exploré exhaustivement. Le MCTS * apporte une réponse élégante à cette question : apprendre à partir de simulations * plutôt que d’explorer toutes les possibilités. Nos contributions montrent qu’il est possible d’améliorer cette approche en intégrant des connaissances stratégiques aux bons endroits.

Nos rollouts guidés * ainsi que notre fonction d’évaluation positionnelle * restent

basés sur des heuristiques simples. Leur intérêt ne vient pas de leur complexité, mais de leur efficacité : orienter les simulations * vers des coups pertinents et évaluer les positions * de manière plus cohérente permet déjà de renforcer sensiblement les performances du MCTS classique. Les résultats expérimentaux confirment cette tendance sur l'ensemble des configurations testées.

Il reste bien sûr des pistes d'amélioration. Notre IA * n'est pas imbattable, et ce n'est d'ailleurs pas l'objectif implicite du projet. Elle joue de manière correcte, voit ses performances s'améliorer avec le budget de calcul, et offre une expérience de jeu cohérente face à un adversaire humain. Elle constitue surtout une base solide pour des améliorations futures et des explorations plus avancées.

Références

- [1] C. Browne *et al.*, *A Survey of Monte Carlo Tree Search Methods*, IEEE Transactions on Computational Intelligence and AI in Games, vol. 4, n° 1, pp. 1–43, 2012. DOI : 10.1109/TCIAIG.2012.2186810
- [2] R. Coulom, *Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search*, Computers and Games, Lecture Notes in Computer Science, vol. 4630, 2006. DOI : 10.1007/978-3-540-75538-8_7
- [3] H. Baier et M. Winands, *Monte-Carlo Tree Search and Minimax Hybrids with Heuristic Evaluation Functions*, Advances in Computer Games, LNCS, vol. 8427, 2014. DOI : 10.1007/978-3-319-14923-3_4
- [4] G. Chaslot *et al.*, *Progressive Strategies for Monte-Carlo Tree Search*, New Mathematics and Natural Computation, vol. 4, n° 03, pp. 343–357, 2008. DOI : 10.1142/S1793005708001094
- [5] J. Schaeffer *et al.*, *Checkers is Solved*, Science, vol. 317, n° 5844, pp. 1518–1522, 2007.
- [6] D. Silver *et al.*, *Mastering the game of Go with deep neural networks and tree search*, Nature, vol. 529, pp. 484–489, 2016.

Annexe A — Constantes globales — utilities.py

Listing 1 – utilities.py — Constantes de pièces et couleurs.

```
1  BLACK_PIECE = 1
2  RED_PIECE   = 2
3  BLACK_KING  = 3
4  RED_KING    = 4
5
6  BROWN      = (139, 69, 19)
7  WHITE      = (255, 255, 255)
8  BLACK      = ( 0,  0,  0)
9  RED        = (255,  0,  0)
10 BLUE       = ( 0,  0, 255)
11 YELLOW     = (255, 255,  0)
```

Annexe B — Point d'entrée — main.py

Listing 2 – main.py — Configuration et lancement du jeu.

```
1  from game.game import Game
2
3  def main():
4      game = Game(mcts_iterations=1500)
5      game.run()
6
7  if __name__ == "__main__":
8      main()
```

Annexe C — Structure d'un nœud MCTS — classe MCTSNode

Listing 3 – MCTSNode — nœud de l'arbre de recherche MCTS.

```
1  class MCTSNode:
2      def __init__(self, board, turn, parent=None, move=None):
3          self.board      = board
4          self.turn       = turn
5          self.parent     = parent
6          self.move       = move
7          self.children   = []
8          self.wins       = 0.0
9          self.visits     = 0
10         self.untried_moves = None
```

```

11
12     def uct_value(self, c=1.41):
13         if self.visits == 0:
14             return float('inf')
15         return (self.wins / self.visits) + c * math.sqrt(
16             math.log(self.parent.visits) / self.visits
17         )
18
19     def best_child(self, c=1.41):
20         return max(self.children, key=lambda ch: ch.
21             uct_value(c))
22
23     def is_fully_expanded(self):
24         return len(self.untried_moves) == 0

```

Annexe D — Rollouts guidés — méthode `_smart_random_move`

Listing 4 – Politique de simulation guidée par heuristiques.

```

1  def _smart_random_move(self, moves, board, turn):
2      captures = [(p, m) for p, ms in moves.items()
3                  for m in ms if abs(m[0] - p[0]) == 2]
4      if captures:
5          return random.choice(captures)
6
7      scored = []
8      center = 3.5
9      for p, ms in moves.items():
10         for m in ms:
11             score = 0
12             if turn == BLACK_PIECE:
13                 score += m[0]
14             elif turn == RED_PIECE:
15                 score += (7 - m[0])
16             score += 2 - abs(m[1] - center) * 0.5
17             if board[p[0]][p[1]] in (BLACK_KING, RED_KING):
18                 score += 1
19             scored.append((score, p, m))
20
21     scored.sort(reverse=True)
22     top = scored[:max(1, len(scored) // 2)]
23     _, p, m = random.choice(top)
24     return p, m

```

Annexe E — Évaluation positionnelle — méthode `evaluate_board`

Listing 5 – Fonction d'évaluation à score continu via sigmoïde.

```

1  def evaluate_board(self, board, mcts_player=None):
2      if mcts_player is None:
3          mcts_player = self.mcts_player
4      red_score = black_score = 0
5
6      for r in range(self.n_squares):
7          for c in range(self.n_squares):
8              p = board[r][c]
9              if p == 0:
10                 continue
11                 center_bonus = 1.0 - abs(c - 3.5) / 7.0
12                 if p == RED_PIECE:
13                     advance = (7 - r) / 7.0
14                     red_score += 10 + 3 * advance +
15                         center_bonus
16                     if r == 7:
17                         red_score += 1
18                 elif p == RED_KING:
19                     red_score += 20 + center_bonus * 2
20                 elif p == BLACK_PIECE:
21                     advance = r / 7.0
22                     black_score += 10 + 3 * advance +
23                         center_bonus
24                     if r == 0:
25                         black_score += 1
26                 elif p == BLACK_KING:
27                     black_score += 20 + center_bonus * 2
28
29             diff = red_score - black_score
30             if mcts_player == BLACK_PIECE:
31                 diff = -diff
32             return 1 / (1 + math.exp(-diff / 15))

```

Annexe F — Boucle de recherche — méthode `mcts_search`

Listing 6 – Orchestration des quatre phases MCTS.

```

1  def mcts_search(self, board, iterations, turn=None):
2      if turn is None:
3          turn = self.mcts_player

```

```

4     root = MCTSNode(board, turn)
5     all_moves = self.get_all_possible_moves(board, turn)
6     root.untried_moves = [(p, m) for p, mvs in all_moves.
7         items()
8             for m in mvs]
9
10    for _ in range(iterations):
11        node = root
12        temp_board = [r[:] for r in board]
13        temp_turn = turn
14
15        while node.untried_moves == [] and node.children:
16            node = node.best_child()
17            if node.move:
18                p, mv = node.move
19                temp_board = self.make_move(temp_board, mv,
20                    p[0], p[1])
21                temp_turn = self._opponent(temp_turn)
22
23        if node.untried_moves:
24            chosen = random.choice(node.untried_moves)
25            node.untried_moves.remove(chosen)
26            p, mv = chosen
27            temp_board = self.make_move(temp_board, mv, p
28                [0], p[1])
29            temp_turn = self._opponent(temp_turn)
30            child = MCTSNode(temp_board, temp_turn,
31                parent=node, move=chosen)
32            child_moves = self.get_all_possible_moves(
33                temp_board, temp_turn)
34            child.untried_moves = [(cp, cm)
35                for cp, cms in
36                    child_moves.items()
37                        for cm in cms]
38            node.children.append(child)
39            node = child
40
41        sim_board = [r[:] for r in temp_board]
42        sim_turn = temp_turn
43        depth = 0
44        while not self.check_winner(sim_board) and depth <
45            80:

```

```
40         sim_moves = self.get_all_possible_moves(  
41             sim_board, sim_turn)  
42         if not sim_moves:  
43             break  
44         p, mv = self._smart_random_move(sim_moves,  
45             sim_board, sim_turn)  
46         sim_board = self.make_move(sim_board, mv, p[0],  
47             p[1])  
48         sim_turn = self._opponent(sim_turn)  
49         depth += 1  
50  
51         result = self.evaluate_board(sim_board, turn)  
52         while node is not None:  
53             node.visits += 1  
54             node.wins += result  
55             node = node.parent  
56  
57     if not root.children:  
58         return None  
59     return max(root.children, key=lambda ch: ch.visits).move
```